

Universidad Nacional de Costa Rica

Sección Regional Huetar Norte y Caribe — Campus Sarapiquí

Trabajo de Investigación Integral

Plataformas y Frameworks Móviles

Sistema académico móvil

Curso: Diseño y Programación de Plataformas Móviles

Docente: Mag. Rachel Bolívar Morales

I Ciclo 2026 | Fecha de entrega: 06 de junio de 2026

Integrantes del equipo:

Natalia Ortiz Martínez

Reichel Corrales Vindas

Keyna Castro Fuentes

Adriana Ortiz Mesén

Docente: Mag. Rachel Bolívar Morales

Resumen

La solución implementa una aplicación móvil híbrida con Ionic y Angular, conectada a un backend REST construido con Node.js y Express, usando Firebase Authentication y Cloud Firestore para autenticación, persistencia y control de acceso por roles.

El sistema permite gestionar usuarios con rol de administrador, profesor y estudiante. Entre las funcionalidades desarrolladas se incluyen login, navegación según rol, gestión de cursos, visualización y eliminación de estudiantes, asignación de estudiantes a cursos, búsqueda en la asignación, registro de evaluación de profesores a los estudiantes los profesores pueden visualizar las evaluaciones por estudiante; los profesores/administradores pueden ver los principales indicadores académicos, gráficos y restricciones de acceso en frontend y backend. Esta base responde a la modularidad, separación de responsabilidades, persistencia, consumo de API, validaciones, estados de carga, retroalimentación visual, trabajo colaborativo con ramas y Pull Requests, y preparación para testing e integración continua.

1. Introducción

El desarrollo móvil moderno exige soluciones que puedan adaptarse a diferentes dispositivos sin duplicar por completo el esfuerzo técnico. En este proyecto se utiliza Ionic como framework principal para construir una aplicación móvil académica con tecnologías web, integrando Angular para la estructura del frontend, servicios REST para la comunicación con backend y Firebase como plataforma de autenticación y base de datos.

La aplicación resuelve una necesidad común en entornos educativos: centralizar la gestión de cursos, estudiantes, profesores y matrículas. A diferencia de un CRUD aislado, el proyecto incorpora roles, navegación diferenciada, autorización, consumo de servicios, validaciones y una interfaz móvil pensada para usuarios administrativos, docentes y estudiantes.

2. Planteamiento del Problema

Las instituciones académicas requieren herramientas que permitan gestionar información de cursos y estudiantes de forma ordenada, segura y accesible. Cuando esta gestión se realiza en hojas de cálculo, mensajes o sistemas no integrados, se incrementa el riesgo de duplicidad

de datos, errores al matricular estudiantes, poca trazabilidad y dificultad para separar permisos entre administradores, profesores y estudiantes.

El problema abordado consiste en diseñar e implementar una solución móvil que permita consultar y administrar información académica según el rol del usuario, manteniendo persistencia en la nube y una arquitectura que facilite evolucionar el sistema hacia nuevas funcionalidades.

3. Desarrollo

3.1 Ionic

Ionic es un framework de código abierto para el desarrollo de aplicaciones móviles híbridas, que fue lanzado en 2013. Utiliza tecnologías web como HTML, CSS y JavaScript para crear aplicaciones nativas para iOS y Android, así como para aplicaciones web progresivas (PWA) (López Senderos, 2023)

Este framework es ideal para el desarrollo de aplicaciones híbridas ya que utiliza tecnologías web estandarizadas, el código utilizado para la creación de aplicaciones en una determinada plataforma también se puede utilizar para la creación de otra app en otra plataforma, con este framework se puede crear aplicaciones que se ejecuten en cualquier plataforma.

Es decir, un framework que nos permite desarrollar aplicaciones para iOS nativo, Android y la web, desde una única base de código. Su compatibilidad y, gracias a la implementación de Cordova e Ionic Native, hacen posible trabajar con componentes híbridos. Se integra con los principales frameworks de frontend, como Angular, React y Vue. (Atmitim et al., 2025)

Entre sus ventajas es que al basarse en tecnologías web, HTML, CSS y JavaScript los desarrolladores no tienen que investigar y aprender nuevas tecnologías además que este se integra con framework habituales como Angular, React y con numerosas herramientas, por último, dispone de numerosos plugins.

El desarrollo de aplicaciones híbridas en un único código propicia un menor tiempo de desarrollo y hace que su mantenimiento y escalado será más sencillo, incluyendo que este presenta interfaces sencillas, donde el desarrollador puede elegir elementos de la UI ya

predeterminados en vez de ir codificando uno por uno. Por último, entre sus ventajas está la buena documentación y respaldo de la comunidad, al ser de código abierto presenta estas ventajas.

Pero, cuáles son sus desventajas: su rendimiento no es el mejor a comparación de las aplicaciones nativas; su dependencia con los plugins esto es una desventaja cuando en una ocasión muy concreta no se encuentren y debemos crearlo nosotros mismos.

3.2 Angular

Angular es un framework cuya arquitectura se basa en componentes como bloques fundamentales de construcción. Los componentes definen las vistas, y utilizan servicios que proveen funcionalidad secundaria no directamente relacionada con la interfaz, los cuales pueden inyectarse como dependencias para mantener el código modular, reutilizable y eficiente. Adicionalmente, Angular distingue los componentes de los servicios con el objetivo de incrementar la modularidad y la reutilización: los componentes se encargan de la experiencia del usuario, mientras que los servicios gestionan tareas como la obtención de datos del servidor o la validación de entradas. (Fain & Moiseev, 2019).

En este sistema, Angular se utiliza para organizar las vistas correspondientes al login, dashboard, cursos, estudiantes y asignación, centralizando la lógica de comunicación con el backend en servicios ubicados dentro de core/services.

3.3 Backend REST con Node.js y Express

Node.js permite ejecutar JavaScript en el servidor gracias a su arquitectura orientada a eventos y su modelo de I/O no bloqueante, lo que lo convierte en una opción eficiente para aplicaciones en tiempo real y servicios web (Doglio, 2018). Por su parte, Express facilita la construcción de rutas HTTP, controladores y middlewares, siendo actualmente el framework más utilizado para el desarrollo de APIs REST sobre Node.js (Doglio, 2018). El backend del proyecto expone endpoints REST para autenticación, cursos, estudiantes, profesores y asignación de estudiantes a cursos, siguiendo los principios de diseño RESTful que establecen recursos identificables mediante URIs únicas y operaciones definidas a través de métodos HTTP estándar.

3.4 Firebase Authentication y Firestore

Firebase Authentication permite gestionar la identidad de usuarios de las aplicaciones, usa contraseñas, número de celular o plataformas como Google y Firebase Admin permite verificar tokens desde el servidor, sin el riesgo que la seguridad esté comprometida (Thomas et al., 2019). Asimismo, Firebase Cloud Firestore es la base de datos en la nube sincronización de manera instantánea, con esquemas flexibles de colecciones/documentos y se implementa para la información de cada usuario (Arias, 2025). La base de datos Firestore utilizada en el proyecto funciona como base de datos NoSQL documental, adecuada para colecciones como users, courses y evaluations. En el sistema, Firestore almacena usuarios, cursos y relaciones de matrícula mediante arreglos de IDs de estudiantes.

3.5 Control de Acceso Basado en Roles

Según Stapleton (2026)

RBAC, o Control de Acceso Basado en Roles, es un modelo de control de acceso ampliamente utilizado para la seguridad que restringe el acceso y la autorización a los sistemas según los roles y responsabilidades de los empleados dentro de la organización. Los permisos RBAC se agrupan por roles, en lugar de asignarse directamente a usuarios individuales. Posteriormente, los usuarios se asignan a sus respectivos roles, lo que simplifica la administración, la gestión de accesos y mejora la seguridad general.

En el proyecto se manejan roles admin, teacher y student. El administrador puede gestionar datos globales; el profesor puede consultar cursos y estudiantes; el estudiante accede a su información académica. Las restricciones se aplican tanto en el frontend con guards como en el backend con middleware.

4. Comparación Técnica: Ionic vs Android Nativo

Criterio	Ionic	Android nativo
Lenguajes	JavaScript, HTML, SCSS y Angular.	Kotlin o Java con XML/Jetpack Compose.
Portabilidad	Alta: una base para web, Android e iOS con Capacitor.	Principalmente Android; iOS requiere otro desarrollo.
Rendimiento	Adecuado para apps empresariales y CRUD; depende del WebView y optimización.	Mayor acceso directo al rendimiento nativo.
UI	Componentes listos y adaptables; fácil personalización con SCSS.	Componentes nativos con integración profunda al sistema.
Curva de aprendizaje	Conveniente para equipos con base web.	Requiere dominio específico del ecosistema Android.
Mantenimiento	Menor duplicación cuando se busca multiplataforma.	Excelente para Android, pero menos reutilizable fuera de Android.

5. Arquitectura del Proyecto

La solución se organiza en dos repositorios: uno móvil y otro backend. El frontend Ionic se comunica con el backend mediante servicios HTTP. El backend valida tokens de Firebase, aplica reglas por rol y consulta Firestore. Esta separación facilita evolucionar cada capa sin mezclar responsabilidades.

Capa	Responsabilidad	Elementos del proyecto
Presentación	Vistas, formularios, tarjetas, tabs y retroalimentación visual.	features/auth, features/dashboard, features/courses, features/students, features/admin-tabs, features/evalautions, features/student-tabs, features/teacher/tabs, features/teachers.
Servicios frontend	Consumo de API y manejo de sesión.	core/services/auth.service.ts, api.service.ts, course.service.ts, student.service.ts
Guardas	Restricción de rutas por rol.	core/guards/auth.guard.ts
Backend REST	Rutas, controladores y validación de permisos.	src/routes, src/controllers, src/middlewares
Persistencia	Documentos NoSQL y autenticación.	Firebase Auth, Firebase Admin, Cloud Firestore

6. Requerimientos Implementados

Requerimiento	Estado en el proyecto
Autenticación	Login integrado con backend y Firebase; tokens verificados en rutas protegidas.
Roles	admin, teacher y student con navegación y permisos diferenciados.
CRUD de cursos	Listar, crear, editar y eliminar cursos desde backend y mobile.
Gestión de estudiantes	Listado, formulario, eliminación y vistas de estudiantes con permisos por rol.

Asignación de estudiantes	Vista con lista multi-selección, búsqueda y actualización del campo students[] del curso.
Registrar Evaluación	Botón de registrar evaluación de un estudiante asociado con el curso, botón de historial para ver la evaluación por estudiante y vista de lista de las evaluaciones del estudiante realizadas por el profesor.
Estados visuales	Loading, empty states, cards, alertas de confirmación y notificaciones.
Trabajo profesional	Uso de ramas, issues, Pull Requests y plantillas de PR por frontend/backend.
Dashboard con métricas	Visualizar los principales indicadores académicos y gráficos

7. Desarrollo Realizado por Módulos

7.1 Autenticación y roles

Se adaptó el flujo de login para retornar usuario, token y rol. El frontend almacena la información de sesión y dirige al usuario a la vista correspondiente. El backend valida credenciales, consulta Firestore y emite/válida tokens. El rol administrativo fue incorporado junto a profesor y estudiante.

7.2 Cursos

El módulo de cursos evolucionó desde una lista visual tipo card hasta un flujo completo de administración. Se agregaron formularios de crear y actualizar cursos, manejo de horarios por día, botón flotante de creación, acciones de editar/eliminar y retroalimentación cuando se elimina un curso.

7.3 Asignación de estudiantes a cursos

Se implementó una pantalla dedicada para ver estudiantes matriculados y asignar nuevos estudiantes. La asignación permite seleccionar múltiples estudiantes, guardar el arreglo de

IDs en `course.students[]`, remover estudiantes y buscar estudiantes antes de asignarlos. La lista se recarga automáticamente después de guardar para mantener la interfaz sincronizada con Firestore.

7.4 Administrador

Se creó la base para el panel administrador. El rol administrativo puede acceder a opciones de gestión y a vistas que permiten administrar cursos, estudiantes y profesores. El login muestra credenciales demo del administrador para facilitar pruebas durante la defensa técnica.

7.5 Profesor y estudiante

El profesor mantiene acceso de consulta a cursos y estudiantes, incluyendo los estudiantes matriculados en cada curso, pero no debe modificar información administrativa si la regla del rol así lo establece. El estudiante accede principalmente a la información que le corresponde, como lista de cursos asignados y datos académicos.

7.6 Evaluaciones

Se implementó un botón en la vista de listar estudiantes la funcionalidad para que el profesor pueda registrar mediante un formulario las evaluaciones asociadas a un estudiante y a un curso en específico, permitiendo calificar la nota, nota máxima, fecha y descripción de esa evaluación. Validación únicamente los usuarios con rol de profesor pueden registrar evaluaciones. Además, se implementó un botón en la vista de listar estudiantes para consultar el historial de un estudiante y los profesor puedan visualizar las evaluaciones de cualquier estudiante.

8. Contratos de API Principales

Método	Endpoint	Descripción
POST	/api/auth/login	Autentica al usuario y retorna token, user y role.
GET	/api/students	Obtener lista de estudiantes
GET	/api/students/:id	Obtener un estudiante en específico
POST	/api/students	Crear un estudiante, Restringido a roles

		autorizados
PUT	/api/students/:id	Actualiza los datos de un estudiante
DELET	/api/students/:id	Eliminar un estudiante en específico
GET	/api/courses	Lista cursos disponibles.
POST	/api/courses	Crea un curso. Restringido a roles autorizados.
PUT	/api/courses/:id	Actualiza los datos de un curso existente.
DELET E	/api/courses/:id	Elimina un curso si existe; retorna 404 si no existe.
PUT	/api/courses/:id/students	Actualiza el arreglo de estudiantes matriculados en un curso.
GET	/api/students	Listas estudiantes para gestión o asignación.
GET	/api/users/teachers	Lista de profesores para el panel administrativo.
POST	/api/users/evaluations	Registrar una nueva evaluación.
GET	/api/evaluations/student/:id	Obtener todas las evaluaciones de un alumno en específico.
GET	/api/dashboard/metrics	Ver métricas y gráficos académicos
GET	/api/dashboard/performance	Obtener una visión general del rendimiento del sistema.
PUT	/api/users/device-token	Guardar el token de notificaciones del celular de cada usuario.

9. Modelo de Datos

Colección	Campos principales	Uso
users	id, email, role, password	Usuarios del sistema: admin, teacher, student.
student	id, name, email, role, career, semester	Estudiantes registrados
courses	id, name, code, credits, schedule, students[]	Cursos y estudiantes matriculados por ID.
evaluations	courseld, title, date, weight, description	Evaluaciones asociadas a cursos, a estudiantes y historial

10. Retos Encontrados y Soluciones

Reto	Solución aplicada
Diferencia entre token personalizado y token verificable por backend	Se ajustó el flujo para persistir y reutilizar el token correcto emitido/aceptado por el backend.
Sincronización después de asignar estudiantes	Se recarga la información del curso/lista después de guardar para reflejar Firestore inmediatamente.
Permisos por rol	Se reforzó la lógica en frontend con guards y en backend con middlewares de rol.
UI de cursos muy cargada	Se rediseñaron tarjetas, acciones y vistas de formulario para separar listar, crear, editar y asignar.
Búsqueda en asignación	Se colocó la búsqueda sobre la lista de resultados para que sea usable aún con muchos estudiantes.
Configuración de Firebase Cloud Messaging en Android	Se configuró el archivo google-services.json y se sincronizó el proyecto con Capacitor para permitir la generación y recepción de tokens FCM en dispositivos Android.
Gestión de tokens de dispositivos	Se implementó un endpoint para almacenar los tokens de

	los dispositivos en Firestore y permitir el envío de notificaciones dirigidas a usuarios específicos
--	--

11. Análisis Crítico del Grupo

Ionic resultó adecuado para una aplicación académica con formularios, navegación por roles, consumo de API y vistas administrativas. Su mayor ventaja fue permitir avanzar con conocimientos web y reutilizar una sola base visual para distintos perfiles de usuario, lo que redujo significativamente el tiempo de desarrollo en comparación con mantener proyectos separados para cada plataforma. Además, la integración nativa con Angular permite aprovechar herramientas ya conocidas como los Guards para el control de rutas, los servicios para centralizar la lógica de negocio y los módulos para organizar el código por dominio funcional

La principal limitación está en que, para funcionalidades muy nativas o de alto rendimiento como el acceso profundo al hardware del dispositivo, procesamiento en segundo plano o animaciones complejas, Android nativo podría ofrecer mayor control y mejor desempeño.. Sin embargo, para el alcance del sistema académico propuesto, Ionic proporciona productividad, componentes móviles listos para usar y suficiente flexibilidad para cubrir los requerimientos planteados sin sacrificar la calidad de la experiencia del usuario.

El proyecto también evidenció que una aplicación móvil real requiere más que pantallas CRUD: autenticación, autorización, manejo de errores, estados visuales, estructura de carpetas, contratos de API, ramas, Pull Requests y pruebas. Cada una de estas capas añade complejidad que debe gestionarse desde el inicio del proyecto, ya que incorporarlas de forma tardía genera deuda técnica difícil de resolver. El trabajo en equipo exige establecer convenciones claras de nombrado, definir responsabilidades por módulo y comunicar los cambios mediante Pull Request, lo que simuló un flujo de trabajo profesional real.

El uso de Inteligencia Artificial apoyó la generación y revisión de ideas, la exploración de alternativas de implementación, Sin embargo, las decisiones finales debieron validarse siempre contra el código existente, el backend disponible y los criterios del enunciado, lo que reafirma la IA es una herramienta de apoyo y no sustituto del criterio técnico del equipo.

Durante el desarrollo del proyecto uno de los principales problemas fue la configuración e integración del capacitador y las notificaciones push, ya que requieren más investigación adicional y pruebas constantes para garantizar el funcionamiento del sistema. Por otra parte, la configuración de Firebase Cloud Messaging y la sincronización de datos al instante, representan desafíos técnicos que requieren validaciones adicionales como también de ajustes.

12. Conclusiones

El desarrollo del sistema académico demuestra que Ionic es una alternativa viable para construir aplicaciones móviles empresariales con enfoque multiplataforma. La combinación Ionic-Angular permite organizar la interfaz por componentes y servicios, mientras que Node.js, Express y Firebase aportan un backend flexible con autenticación y persistencia en la nube, conformando un stack moderno que puede adaptarse a distintos contextos y escalas de proyecto.

La implementación de roles admin, teacher y student fortalece la seguridad funcional del sistema, ya que cada usuario observa o modifica únicamente la información que corresponde a sus permisos, reduciendo el riesgo de accesos indebidos y manteniendo la integridad de los datos. Este esquema de autorización basado en roles demostró ser uno de los pilares más importantes del diseño, ya que sin él la aplicación no tendría sentido como sistemas académicos reales.

La gestión de cursos y la asignación de estudiantes evidencian una solución práctica al problema económico planteado, con posibilidades claras de crecimiento hacia evaluaciones, notificaciones push, carga de archivos y analítica de rendimiento académico. El sistema fue construido con esa escalabilidad en mente, lo que se refleja en la separación de responsabilidades entre frontend y backend, y en la estructura modular del proyecto.

Finalmente, el grupo concluye que el mayor aprendizaje no fue técnico sino metodológico: planificar antes de codificar, definir contratos de API antes de implementar pantallas, y validar cada decisión contra los requerimientos reales del sistema. Estos hábitos, más que cualquier tecnología específica, son los que determinan el éxito de un proyecto de software en un entorno profesional

13. Bibliografía

Atmitim, J. M. A., Maluenda, R., Atmitim, J. M. A., & Maluenda, R. (Septiembre 11, 2025). *Qué es Ionic: ventajas y desventajas de usarlo para desarrollar apps móviles híbridas*. Profile Software Services. <https://profile.es/blog/que-es-ionic/>

Doglio, F. (2018). *REST API Development with Node.js: Manage and Understand the Full Capabilities of Successful REST Development*. pg 123 Apress / Springer Nature. https://books.google.co.cr/books?hl=es&lr=&id=kjUwCgAAQBAJ&oi=fnd&pg=PR7&dq=REST+API+Development+with+Node.js&ots=f3a1HtaSxc&sig=3gZ9PKCS_2LyvM5jFx_E7At8InuM&redir_esc=y#v=onepage&q=REST%20API%20Development%20with%20Node.js&f=false

Fain, Y., & Moiseev, A. (2019). *Angular Development with TypeScript* (2nd ed.). Manning Publications. <https://www.manning.com/books/angular-development-with-typescript-second-edition>

Google LLC. (2024). *Introduction to Angular concepts*. Angular Documentation. <https://v17.angular.io/guide/architecture>

López Senderos, L. (Febrero 20, 2023). *Ionic: ¿Qué es? ¿Es el futuro de las apps?* | SEIDOR. Ionic: ¿Qué es? ¿Es El Futuro De Las Apps? | SEIDOR. <https://www.seidor.com/es-cr/blog/ionic-que-es-es-el-futuro-de-las-apps>

Stapleton, S. (Marzo 26, 2026). Role-Based Access Control (RBAC) – a comprehensive guide. *Pathlock*. <https://pathlock.com/blog/role-based-access-control-rbac/>

Arias, P. (2025, junio). *Desarrollo de una Aplicación Móvil para la Gestión Personalizada de Hábitos*. Archivo Digital UPM | Archivo Digital UPM. https://oa.upm.es/90079/1/TFG_PAULA_ARIAS_CABEZAS.pdf

Thomas, J., Carbonell, & Lloret, J. (2019). Firebase: trabajar en la nube. En *Google Books*. Alpha Editoria. <https://www.marcombo.com/libro/libros-tecnicos-de-arte-y-cientificos/informatica-libros-tecnicos-y-cientificos/otros-informatica/firebase-trabajar-en-la-nube/>